

Góp ý bổ sung

Nội dung chương trình DevOps

1. Lecture 1: GitHub

Link tài liệu:

https://drive.google.com/file/d/1xsXSaGJ4ErXzE3asZijlmU_LWaXpYi3o/view?usp=sharing

Link dự án gốc: <https://github.com/ht-canh/devops-project-hcm>

Bổ sung các mục:

- Hướng dẫn cơ bản về cấu trúc dự án Node.js, Express, React
- Cách triển khai Database (ví dụ với Postgres)
- Cách chạy và bug lỗi dự án trong môi trường phát triển
- Xây dựng Test case

1.1. Hướng dẫn sử dụng GitHub

- Fork và Clone code (Nếu làm theo hướng dẫn)
- Tạo Repository và đẩy code lên hệ thống (chỉnh sửa khai báo git add . trong hướng dẫn từ gitHub để cho phép đẩy toàn bộ code từ máy local lên)
- Mời developer vào team.
- Một số câu lệnh git phổ biến
 - Git clone [url]: tải code về máy local
 - Git checkout [branch]: Chuyển nhánh
 - Git checkout -b [new-branch]: tạo nhánh mới
 - Git status: Kiểm tra trạng thái ở nhánh hiện tại (xem lại các file nào đã bị thay đổi)
 - Git add [files]: Đưa các file đã thay đổi vào Stage

- Git commit -m "[message]": Thừa nhận đã thay đổi với nội dung trong message.
- Git push [branch]: Đẩy code vào nhánh [branch]. Nếu không có [branch], code sẽ được đẩy vào nhánh hiện tại
- Git pull: Kéo code của nhánh hiện tại về máy local

1.2. Chạy dự án:

Khai báo file .gitignore cho cả 2 dự án và push lên git hub. Đây là các file cần phải thực hiện đầu tiên trước khi thực hiện các lệnh của git hub. Tuy nhiên, thư mục dự án của bạn khi tải về có thể đã được liên kết với tài khoản GitHub của developer khác. Bạn cần thực hiện các bước sau:

- Kiểm tra trạng thái đăng nhập: gh auth status
- Xóa thư mục ẩn .git: rm -rf .git
- Note: có thể xóa trực tiếp thư mục .git trong dự án thay cho câu lệnh trên

Cài đặt packages cho 2 dự án: npm install

Tạo lập biến môi trường cho cả 2 dự án:

- cài đặt dotenv để nhận dạng process.env.[Key] `npm install dotenv`
- tạo các file .env, .gitignore,

Dự án Front End:

tạo file index.js trong src. Đây là file cần có và sẽ là file quan trọng thứ 2 được gọi đến trong dự án React (sau file index.html)

```
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

Tạo file public/index.html với nội dung. Bắt buộc phải có thẻ `<div id="root"></div>`

```
<!doctype html>
<html lang="en">

<head>
  <meta charset="utf-8" />
  <link rel="icon" href="/favicon.ico" />
  <meta name="viewport" content="width=device-width,initial-scale=1" />
  <meta name="theme-color" content="#000000" />
  <meta name="description" content="Web site created using create-react-app" />
  <link rel="apple-touch-icon" href="/logo192.png" />
  <link rel="manifest" href="/manifest.json" />
  <title>React App</title>
  <script defer="defer" src="/static/js/main.5bcfeb08.js"></script>
</head>

<body>
  <noscript>You need to enable JavaScript to run this app.</noscript>
  <div id="root"></div>
</body>

</html>
```

Tạo file .env trong thư mục frontend với nội dung sau

```
REACT_APP_API_URL=http://localhost:5000
```

Build dự án front end: ***npm run build***

Việc build dự án front end không nhất thiết thực hiện. Có thể bổ sung 1 câu lệnh script start trong file package.json để chạy lệnh ***npm run dev*** như sau

```
"scripts": {
  ...
  "dev": "react-scripts start",
  ...
},
```

Dự án Backend:

Cài đặt dotenv với npm install dotenv

Tạo file .env với nội dung sau

```
PORT=3000
DB_HOST=localhost
DB_NAME=tododb
DB_USER=postgres
DB_PASSWORD=123456789
DB_PORT=5432
JWT_SECRET=your_jwt_secret
```

Thêm dòng sau vào dòng đầu tiên của file server.js

```
require('dotenv').config();
```

Fix các bug của dự án (Hãy hỏi ChatGPT để Fix)

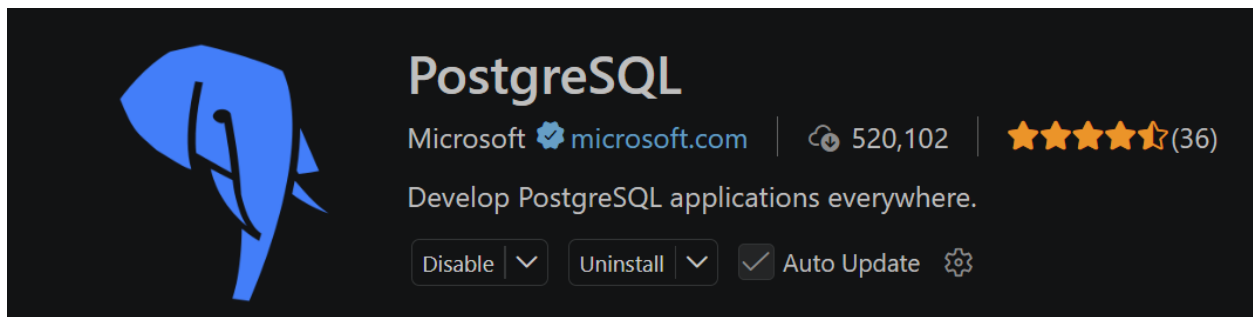
Chạy 2 dự án với câu lệnh ***npm start***

Cài đặt PostgreSQL trên Docker

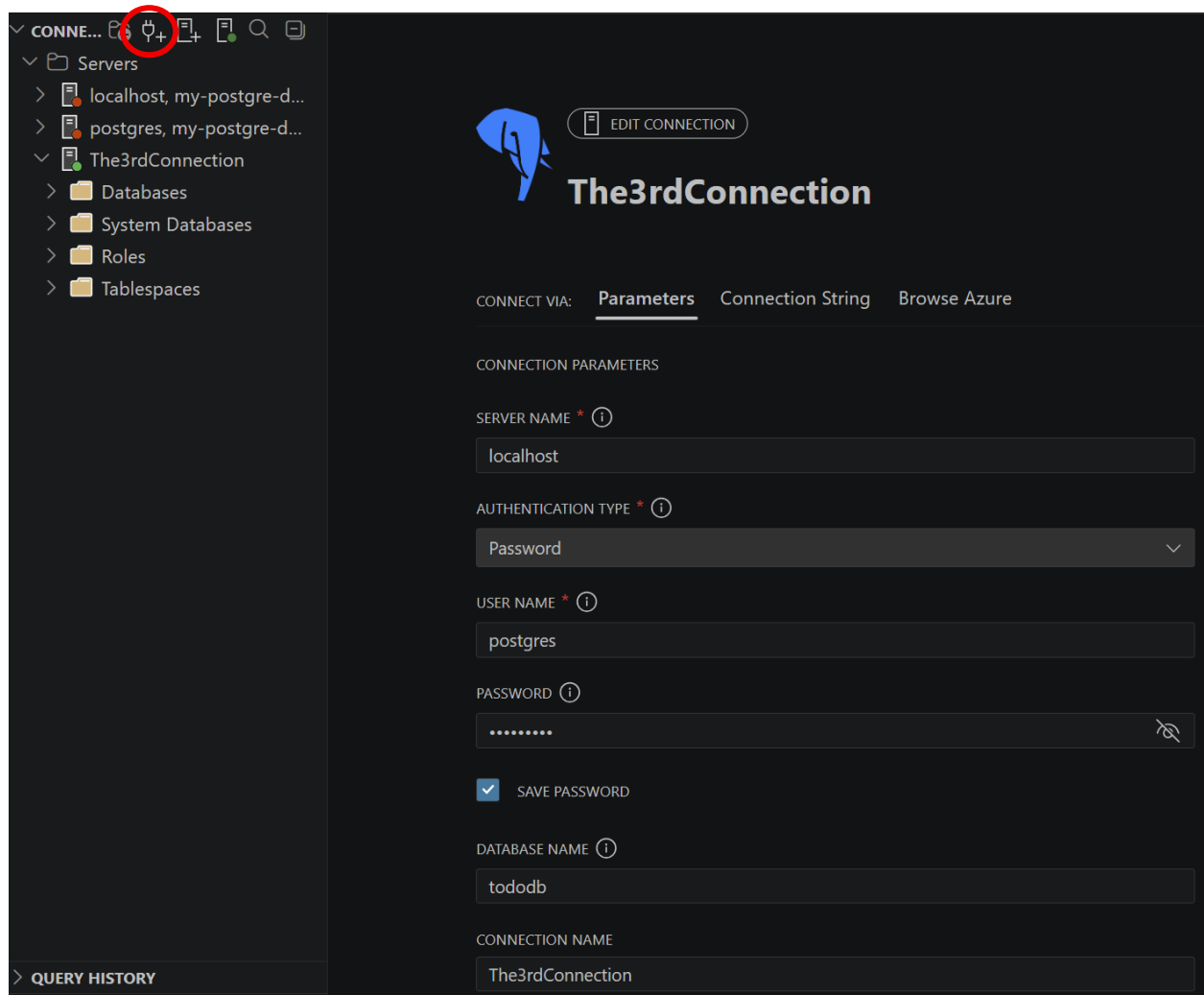
- Tải image về docker: `docker pull postgres`
- Chạy container

```
docker run -d --name postgres-container -e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=123456789
-e POSTGRES_DB=tododb -p 5432:5432 postgres:latest
```

Cài Extension PostgreSQL của Microsoft trong VS Code



Thực hiện thao tác kết nối với các thông số đã triển khai



Lưu ý:

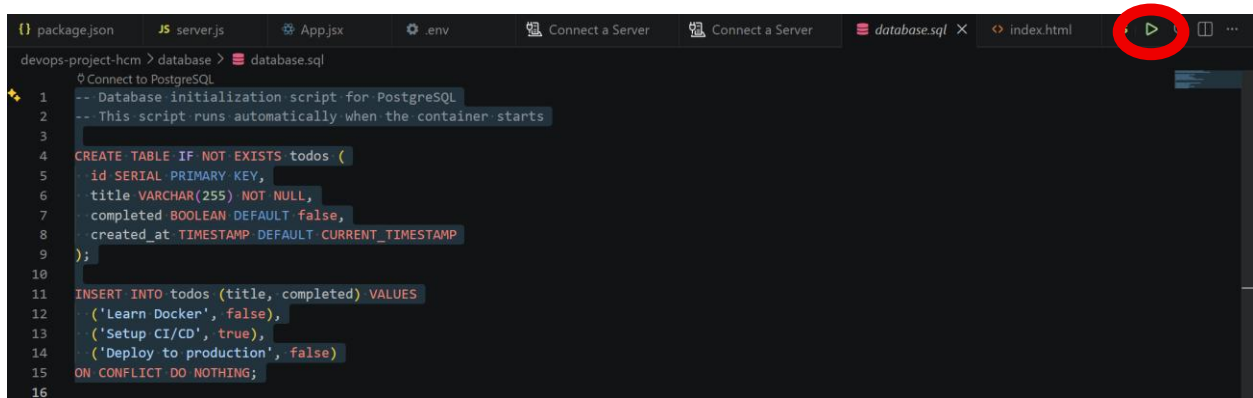
- Phải đảm bảo Docker đã được khởi động
- Đảm bảo container đã được chạy đúng như câu lệnh trên
- Nếu có nhiều container cùng port (hoặc đã cài đặt PostgreSQL local) thì phải vô hiệu hóa trước khi chạy

Data Seeding: Sinh dữ liệu Database

Truy cập file database/database.sql

Chọn các lệnh cần chạy

Bấm nút Tam giác màu xanh lá bên phải



```
1 -- Database initialization script for PostgreSQL
2 -- This script runs automatically when the container starts
3
4 CREATE TABLE IF NOT EXISTS todos (
5   id SERIAL PRIMARY KEY,
6   title VARCHAR(255) NOT NULL,
7   completed BOOLEAN DEFAULT false,
8   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
9 );
10
11 INSERT INTO todos (title, completed) VALUES
12   ('Learn Docker', false),
13   ('Setup CI/CD', true),
14   ('Deploy to production', false)
15 ON CONFLICT DO NOTHING;
```

Về sau khi triển khai Docker, chúng ta sẽ khai báo thêm dòng sau trong khối lệnh volume trong phần khai báo database của postgres của file docker-compose.yml (như 2 phần bên dưới). Docker sẽ tự động chạy các lệnh được đặt sẵn trong file database.sql

```
volumes:
  - postgres_data:/var/lib/postgresql/data
  - ./database/database.sql:/docker-entrypoint-initdb.d/database.sql
```

```
postgres:
  image: postgres:15-alpine
  # TODO: Add environment, ports, volumes, healthcheck
  environment:
    POSTGRES_DB: tododb
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: 123456789
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./database/database.sql:/docker-entrypoint-initdb.d/database.sql
```

2. Lecture 2: Docker

Lần lượt tạo các file .dockerignore và Dockerfile trong 2 thư mục dự án frontend và backend. Lưu ý: các file phải giống tên như trên, khi tạo thành công có biểu tượng Docker bên tay trái

- File .dockerignore: Quy ước những file mà Docker sẽ bỏ qua khi tạo ảnh Image. Lưu ý: do các dự án triển khai phụ thuộc vào biến môi trường như

process.env.NODE_ENV nên chúng ta **không** được khai báo .env trong file .dockerignore. Nội dung như sau:

```
node_modules
# production build
build
dist
# environment variables
<!-- .env -->
# editor files
.vscode
.idea
# OS files
.DS_Store
Thumbs.db
# git
.git
.gitignore
# Docker
Dockerfile
.dockerignore
```

- Dockerfile: chứa những khối lệnh mà Docker sẽ căn cứ vào khi triển khai tạo Image

Tạo Docker cho dự án backend: Tạo nội dung như bên dưới rồi chạy 2 câu lệnh sau trong Terminal

- cd backend
- docker build -t backend-todo-app:1.1 .
- docker run -d --name backend-app-container -p 5000:5000 backend-todo-app:1.1
- Nội dung:

```
# Stage 1: Builder
# =====
FROM node:18-alpine AS builder

# Set working directory
WORKDIR /app

# Copy package files
```

```

COPY package*.json ./

# Install all dependencies
RUN npm install

# Copy source code
COPY . .

# =====
# Stage 2: Production
# =====
FROM node:18-alpine

# Set working directory
WORKDIR /app

# Copy package files from builder
COPY --from=builder /app/package*.json ./

# Install only production dependencies
RUN npm install --only=production

# Copy application code from builder
COPY --from=builder /app ./

# Expose application port
EXPOSE 5000

# Start the application
CMD ["npm", "start"]

```

Lưu ý: Do image của postgres đã được tạo trước đó bằng câu lệnh sau:

```

docker run -d --name postgres-container -e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=123456789
-e POSTGRES_DB=tododb -p 5432:5432 postgres:latest

```

container được sinh ra có tên là postgres-container, Để Container của dự án Backend có thể kết nối với Container chứa Postgres bạn cần phải thực hiện các lệnh sau (về sau sẽ triển khai docker-compose)

- Tạo một Docker network: `docker network create my-app-network`

- Kết nối container Postgres hiện tại vào network vừa tạo (phải đảm bảo container chứa postgres của bạn đã được chạy với tên postgres-container):
docker network connect my-app-network postgres-container
- Build Image cho dự án backend: docker build -t backend-todo-app .
- Chạy Container với các tham số môi trường truyền vào như sau: docker run -d --name backend-app-container --network my-app-network -p 5000:5000 -e DB_HOST=postgres-container -e DB_PORT=5432 -e DB_USER=postgres -e DB_PASSWORD=123456789 -e DB_NAME=tododb backend-todo-app

Tạo Docker cho dự án frontend. Tạo nội dung như bên dưới rồi chạy 2 câu lệnh sau trong Terminal

- cd frontend
- docker build -t frontend-todo-app:1.1 .
- docker run -d --name frontend-todo-app-container -p 3000: 3000 frontend-todo-app:1.1

```
# --- Stage 1: The "builder" stage ---
# Use a full Node.js image to build our app
FROM node:18-alpine AS builder
# Set the working directory inside the container
WORKDIR /app
# Copy package.json and package-lock.json (or yarn.lock)
COPY package*.json ./

# Install all dependencies (including devDependencies)
RUN npm install

# Copy the rest of the application source code
COPY . .
# (If your project has a build step, e.g., TypeScript, add it here)
RUN npm run build

# --- Stage 2: The "production" stage ---
# Start from a fresh, lightweight base image
FROM node:18-alpine
WORKDIR /app

# Copy the package.json and install *only* production dependencies
```

```

COPY package*.json ./
RUN npm install --production # or npm install --only=production

# Copy the built app code from the "builder" stage
COPY --from=builder /app/src ./src

# (If you had a build step, you would copy the 'dist' folder instead)
# COPY --from=builder /app/dist ./dist
COPY --from=builder /app/build ./build
# Expose the port the app listens on (check your app's code, e.g., server.js)
EXPOSE 3000

# The command to run the application (check your package.json 'start' script)
CMD [ "npm", "start" ]

```

Lưu ý: Khác biệt của các file Dockerfile trong 2 dự án ở chỗ:

- Dự án frontend chạy React đòi hỏi phải build bằng lệnh RUN npm run build
- Copy file dự án sau build sẽ có cấu trúc đường dẫn khác

Sau khi chạy các câu lệnh docker, trên ứng dụng Docker Desktop sẽ có những Image và Container tương ứng.

Triển khai Docker Compose

Tạo file có tên docker-compose.yml trong thư mục mẹ, ngang cấp với 2 thư mục frontend và backend. Bạn cũng có thể tạo thêm file :

- docker-compose.override.yml để ghi đè trong trường hợp thiết lập môi trường riêng là development hoặc production,
- hoặc docker-compose.prod.yml để ghi đè cho trường hợp production

Quy tắc cần nắm trước khi triển khai :

- Khai báo mục con sẽ được thụt đầu dòng 1 cấp so với mục mẹ
- Port Mapping: được khai báo trong mục ports với định dạng [Host port] : [Docker port]
- Dependency : Khai báo phụ thuộc sẽ được khai báo trong mục depends_on:
 - Dự án FrontEnd (frontend) sẽ phụ thuộc vào dự án Backend (backend)
 - Dự án Backend sẽ phụ thuộc vào Database (postgres)

- Database: Do các database sẽ được triển khai trên Docker nên ta không thể khai báo DB_HOST là localhost được (nếu vậy docker sẽ hiểu là chính dự án backend). Phải khai báo theo tên của Database trong phần khai báo database của file docker-compose.yml (trong khai báo bên dưới nó là postgres)
- Volume: khai báo ổ đĩa lưu trữ dữ liệu cho database. Nếu không khai báo, nếu người dùng Xóa Container của Docker là toàn bộ dữ liệu sẽ bị mất.

Hướng dẫn triển khai: Hỏi ChatGPT với context được copy từ nội dung gợi ý của giáo viên ở trong file docker-compose.yml đã tạo sẵn, kèm theo Node Express ngay dòng Backend và React ngay dòng FrontEnd

Nội dung file docker-compose.yml như sau:

```
version: '3.8'

services:
  # STUDENT TODO: Define frontend service
  frontend:
    build: ./frontend
    # TODO: Add ports, depends_on
    ports:
      - "3000:3000"
    depends_on:
      - backend

  # STUDENT TODO: Define backend service
  backend:
    build: ./backend
    # TODO: Add ports, environment, depends_on, healthcheck
    ports:
      - "5000:5000"
    environment:
      DB_HOST: postgres
      DB_PORT: 5432
      DB_NAME: tododb
      DB_USER: postgres
      DB_PASSWORD: 123456789
    depends_on:
      postgres:
        condition: service_healthy
```

```

healthcheck:
  test: ["CMD", "wget", "--spider", "-q", "http://localhost:5000/health"]
  interval: 10s
  timeout: 5s
  retries: 5
  start_period: 15s
# STUDENT TODO: Define postgres service
postgres:
  image: postgres:15-alpine
  # TODO: Add environment, ports, volumes, healthcheck
  environment:
    POSTGRES_DB: tododb
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: 123456789
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./database/database.sql:/docker-entrypoint-initdb.d/database.sql
  healthcheck:
    test: ['CMD-SHELL', 'pg_isready -U postgres']
    interval: 10s
    timeout: 5s
    retries: 5

# STUDENT TODO: Define named volumes
volumes:
  postgres_data:

```

Thực thi lệnh trên Docker:

- Tạo Image và chạy Container: `docker-compose up -d --build`
- Xem nhật ký: `docker-compose logs -f`
- Dừng và Xóa Container: `docker-compose down`
- Chạy lại Image đã build trước đó: `docker-compose up -d`

Test thử: Mở Docker Desktop, vào mục Container và bấm vào vòng tròn đỏ bên dưới để mở Browser.

☐	▼	●	devops-project-hcm	-	-	-			
☐		●	frontend-1	f1270e1b82c1	devops-pro	3000:3000			
☐		●	backend-1	9ae1d1d68bfb	devops-pro	5000:5000			
☐		●	postgres-1	a5e5144ca416	postgres:1				

Mapping Port trong Dockerfile phân biệt rõ 02 trường hợp

- Development (cần chạy thử để biết Port thật trên máy Dev)

Trong file docker-compose.override.yml (development)

```
version: "3.8"
services:
  web:
    build: .
    ports:
      - "3000:3000"
    volumes:
      - ../app
    environment:
      - NODE_ENV=development
```

- Production: mặc định dùng Port 80, hoặc 8080 trên Host

Trong file docker-compose.prod.yml (production)

```
version: "3.8"
services:
  web:
    image: myapp:latest
    ports:
      - "8080:8080"
    environment:
      - NODE_ENV=production
    restart: always
```

Kết hợp với việc chạy lệnh: `docker-compose -f docker-compose.yml -f docker-compose.prod.yml up -d`.

Hướng dẫn thống nhất sử dụng lệnh: npm ci thay cho toàn bộ các lệnh npm install (các slide)

Thực thi lệnh Create Table + Insert Data: trong file database.sql (thư mục database): trong phần khai báo docker-compose phần database. Dòng lệnh: - ./database.sql:/docker-entrypoint-initdb.d/database.sql

```
postgres:
  # image: postgres:latest
  image: postgres:15-alpine
  # TODO: Add environment, ports, volumes, healthcheck
  # image: postgres:16
  container_name: postgres_db
  restart: always

  environment:
    POSTGRES_USER: admin
    POSTGRES_PASSWORD: 12345678
    POSTGRES_DB: my-postgre-db

  ports:
    - "5432:5432"

  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./database.sql:/docker-entrypoint-initdb.d/database.sql
```

3. Lecture 3: Triển khai CI với GitHub Action

3.1. GitHub Action Introduction

Giới thiệu một số GitHub **Action** phổ biến như:

- actions/checkout
 - Checkout code từ repository vào runner (máy ảo CI/CD).
 - Hỗ trợ:
 - Lấy source code của branch, tag, hoặc commit cụ thể.
 - Làm việc với submodules và LFS (Large File Storage).
 - Tùy chỉnh độ sâu clone (`fetch-depth`) để tăng tốc.
- actions/setup-node
 - Cài đặt phiên bản Node.js mong muốn (ví dụ: 16, 18, 20).
 - Hỗ trợ cache dependencies (`npm`, `yarn`, `pnpm`) để tăng tốc CI/CD.
 - Tự động thêm Node.js vào `PATH` để các lệnh như `node`, `npm` có thể chạy ngay.

- Cho phép chọn registry và cấu hình authentication cho npm.
- **docker/login-action**
 - Đăng nhập vào Docker registry bằng username và password/token.
 - Thiết lập thông tin xác thực để các bước sau có thể **docker build** và **docker push**.
 - Hỗ trợ nhiều registry khác nhau (Docker Hub, GHCR, Azure Container Registry...).
- **docker/build-push-action**
 - Build Docker image từ Dockerfile.
 - Push image lên registry (Docker Hub, GitHub Container Registry, Amazon ECR, v.v.).
 - Hỗ trợ multi-platform builds (qua Buildx).
 - Cho phép tag nhiều phiên bản của image trong cùng một bước.
 - Tích hợp với cache để tăng tốc độ build.
- **aws-actions/configure-aws-credentials**
 - Thiết lập AWS access key, secret key, và session token từ GitHub Secrets.
 - Cấu hình AWS region để các lệnh CLI/SDK biết nơi thực thi.
 - Tích hợp với OIDC (OpenID Connect) để cấp quyền tạm thời mà không cần lưu trữ access key lâu dài.
 - Tự động export biến môi trường **AWS_ACCESS_KEY_ID**, **AWS_SECRET_ACCESS_KEY**, **AWS_SESSION_TOKEN**, và **AWS_REGION**.

Giới thiệu một số **Toolkits** phổ biến như:

- **@actions/core**
 - Cung cấp API cơ bản để:
 - Thiết lập và lấy đầu ra (**setOutput**, **getInput**)
 - Ghi log (**info**, **warning**, **error**)
 - Quản lý trạng thái workflow (**setFailed**)
 - Ứng dụng: Hầu hết mọi custom Action đều cần **core**.
- **@actions/github**
 - Wrapper cho GitHub REST API và context của workflow.

- Cho phép:
 - Lấy thông tin repo, commit, pull request
 - Tương tác với GitHub API dễ dàng
- Ứng dụng: Tự động tạo issue, comment vào PR, trigger workflow khác.
- **@actions/exec**
 - Chạy lệnh shell trong Action.
 - Hỗ trợ:
 - Chạy command-line tool
 - Bắt output và xử lý kết quả
 - Ứng dụng: Build, test, deploy bằng script tùy chỉnh.
- **@actions/io**
 - Quản lý file và thư mục:
 - Copy, move, delete
 - Tạo thư mục cần thiết
 - Ứng dụng: Chuẩn bị môi trường trước khi chạy CI/CD.
- **@actions/tool-cache**
 - Cache và quản lý binary/tool.
 - Cho phép:
 - Tải về và lưu trữ tool để tái sử dụng
 - Giảm thời gian build
 - Ứng dụng: Cài đặt Node, Python, Go... nhanh hơn.

3.2. Các thiết lập Settings: Rules

- Chỉ cho phép những người được mời vào Team push code lên các nhánh
- Chỉ cho phép Merge code khi có ít nhất một Approved Pull Request
- Chỉ cho phép chạy CD ở nhánh production (nâng cao)

3.3. Thiết lập Credentials (Secrets)

Cấu hình Repository Secrets: được dùng để tạo các thông số kết nối đến máy chủ Server (khi triển khai CD bạn cần chuẩn bị thông số máy chủ thật, trong giai đoạn CI hãy sử dụng thông số trên máy local dev). Các bước thực hiện như sau:

- Actions - Secrets and Variables - Repository Secrets
- Tạo các Key HOST, PORT, PASSWORD, USERNAME
- Đính vào trong file .github/workflow/main.yml (hoặc ci.yml)
\${{secrets.HOST}}, \${{secrets.PORT}}, etc.

3.4. Docker Hub

Ta cần publish các image đã tạo trên Docker lên Docker Hub để Git Hub Actions có thể tương tác trực tiếp. Bước này chỉ thực hiện khi sử dụng các image được build manually. Trong hướng dẫn của Lab, bạn chỉ cần khai báo Username và Password của DockerHub và gọi dòng lệnh uses: docker/build-push-action@v5 trong file ci.yml

- Bước 1: Đăng ký tài khoản Docker Hub ở trang <https://hub.docker.com/> (Lưu ý ghi nhớ username không phải email đăng nhập)
- Bước 2: Đăng nhập trên Terminal bằng lệnh docker login

Cấu trúc tên repository trên Docker Hub:

<username>/<repository_name>:<tag>

Build image: chạy lệnh theo các cấu trúc sau:

- Nếu chưa có image: docker build -t
<username_cua_ban>/<ten_repository>:<tag_phien_ban> .

```
docker build -t riolish/todo-web-app:v1.0 .
```

- Nếu đã từng build image với tên backend-todo-app trước đó: docker tag
<ten_image_cu>
<username_cua_ban>/<ten_repository>:<tag_phien_ban>

```
docker tag backend-todo-app riolish/todo-web-app:v1.0
```

Push image lên Docker Hub: (Không cần thực thi trong thực tế)

- Cấu trúc lệnh: docker push
<username_cua_ban>/<ten_repository>:<tag_phien_ban>
- Thực thi trong Terminal:

```
docker push riolish/todo-web-app:v1.0
```

3.5. Triển khai ci.yml

Vị trí: Tạo file ci.yml trong thư mục sau: .github/workflows/ci.yml

Các lưu ý:

- Tạo thư mục triển khai trên Server VPS: Phải tạo trước thư mục /var/wwwrootdev1/devops-project-hcm-cos1203
- Khai báo Secrets trên GitHub cho Repository
- Tạo tài khoản DockerHub tại <https://hub.docker.com/>
- Đảm bảo các Image của Docker local phải chạy được trên thực tế (các Dockerfile hoạt động)

Nội dung file mẫu:

```
name: CI/CD Pipeline

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  # TESTING BACKEND
  test:
    name: Run Backend Tests
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:15-alpine
        env:
          POSTGRES_USER: admin
          POSTGRES_PASSWORD: 12345678
          POSTGRES_DB: my-postgre-db
        ports:
          - 5438:5432
        options: >-
```

```

    --health-cmd "pg_isready -U admin -d my-postgre-db"
    --health-interval 10s
    --health-timeout 5s
    --health-retries 5

steps:
  - name: Checkout Code
    uses: actions/checkout@v4

  - name: Initialize Database
    env:
      # psql automatically looks for this environment variable for the
password
      PGPASSWORD: 12345678
    run: |
      until pg_isready -h 127.0.0.1 -p 5438 -U admin -d my-postgre-db; do
        echo "Waiting for Postgres to open TCP connections..."
        sleep 2
      done

      psql -h 127.0.0.1 -p 5438 -U admin -d my-postgre-db -f
./database/database.sql

  - name: Setup Node.js
    uses: actions/setup-node@v4
    with:
      node-version: '24'

  - name: Install Dependencies
    run: cd backend && npm ci

  - name: Run Tests
    env:
      DB_USER: admin
      DB_PASSWORD: 12345678
      DB_HOST: localhost
      DB_PORT: 5438
      DB_NAME: my-postgre-db
    run: cd backend && npm test

# BUILD & PUSH DOCKER IMAGES
build-and-push:
  name: Build and Push Images
  needs: test # This ensures tests MUST pass before building
  runs-on: ubuntu-latest

```

```
# We only want to build and push when code is merged to main, not on PRs
if: github.event_name == 'push' && github.ref == 'refs/heads/main'

steps:
  - name: Checkout Code
    uses: actions/checkout@v4

  - name: Log in to Docker Hub
    uses: docker/login-action@v3
    with:
      username: ${ secrets.DOCKERHUB_USERNAME }
      password: ${ secrets.DOCKERHUB_TOKEN }

  - name: Build and Push Backend
    uses: docker/build-push-action@v5
    with:
      context: ./backend
      push: true
      tags: ${ secrets.DOCKERHUB_USERNAME }/devops-backend-cos1203:latest

  - name: Build and Push Frontend
    uses: docker/build-push-action@v5
    with:
      context: ./frontend
      push: true
      tags: ${ secrets.DOCKERHUB_USERNAME }/devops-frontend-cos1203:latest
```

4. Lecture 4: Triển khai CD

4.1. Chuẩn bị trên Ubuntu Server (VPS)

Tạo thư mục triển khai trên Server VPS:

- Phải tạo trước thư mục /var/www/devops-project-hcm-cos1203
- Lệnh thực thi: `sudo mkdir -p /var/www/devops-project-hcm-cos1203`
- Clone code về thư mục này (thực thi sau khi code hoàn chỉnh):
 - o `cd /var/www/devops-project-hcm-cos1203`
 - o `sudo git clone https://github.com/Team Leader/devops-group-1.git`

Cấu hình key trên server:

- Tạo cặp khóa: `ssh-keygen -t rsa -b 4096 -m PEM -f my-rsa-key.pem`
- Phân quyền: `chmod 400 my-rsa-key.pem`
- Liệt kê các khóa: truy cập vào thư mục `.ssh` bằng lệnh `cd .ssh` và liệt kê bằng lệnh `ls`
- Copy về máy local: `scp username@server-ip:~/.ssh/my-rsa-key.pem .`
- Đọc file public key: bằng lệnh `cat id_rsa.pub`
- Truy cập server thông qua private key: `ssh -i /path/to/your/id_rsa.pem username@server-ip-address` (đường dẫn đến file pem có thể là `~/.ssh/my-rsa-key.pem`)

Lưu ý:

- Phân biệt private và public keys : thông thường public key được lưu trữ trong các file có đuôi `.pub` như ví dụ trên nó sẽ là `my-rsa-key.pem.pub`
- Để an toàn, người ta thường sử dụng public key thay vì private key như hướng dẫn của Lab

4.2. Bổ sung GitHub Action Secrets

- Bổ sung các Repository Secrets còn thiếu
 - HOST
 - USERNAME
 - SSH_PRIVATE_KEY
 - TEST_DB_PASSWORD và các database Test
- Cấu hình public key trên GitHub
 - Vào Settings (Account trên GitHub)
 - Chọn SSH and GPG keys
 - Bấm New SSH key
 - Title: `project_deploy_public_key`
 - Key: Dán nội dung vào phần value
- Kiểm tra kết nối từ Server lên Github
 - Kết nối ssh
 - Gõ lệnh `ssh -T git@github.com`
-

4.3. Cập nhật file `docker-compose.yml`

- Bổ sung các khai báo image và container ở tất cả các services
- Cho PostgreSQL

```
postgres:
  image: postgres:15-alpine
  container_name: devops-postgres-cos1203
  env_file:
    - .env
  environment:
    POSTGRES_USER: ${DB_USER}
    POSTGRES_PASSWORD: ${DB_PASSWORD}
    POSTGRES_DB: ${DB_NAME}
  # IMPORTANT: Do not map ports like 5433:5432 to keep the DB internal
  volumes:
    - postgres_data:/var/lib/postgresql/data
    - ./database/database.sql:/docker-entrypoint-initdb.d/database.sql
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
    interval: 5s
    timeout: 5s
    retries: 5
```

- Cho frontend

```
# STUDENT TODO: Define frontend service
frontend:
  build:
    context: ./frontend
    dockerfile: Dockerfile
  image: [docker_image_published_from_DockerHub]:latest
  container_name: devops-frontend-cos1203
  ports:
    - 3010:3000
  depends_on:
    - backend
```

- Cho backend

```
backend:
  image: [docker_image_published_from_DockerHub]:latest
  container_name: devops-backend-cos1203
  ports:
    - "8081:8080"
```

```
env_file:
  - .env
depends_on:
  postgres:
    condition: service_healthy
```

4.4. Cập nhật file ci.yml

- Bổ sung nội dung file ci.yml (hoặc main.ci)
- Cho phần test:

```
test:
  name: Run Backend Tests
  runs-on: ubuntu-latest
  services:
    postgres:
      image: postgres:15-alpine
      env:
        POSTGRES_USER: postgres
        POSTGRES_PASSWORD: ${ secrets.TEST_DB_PASSWORD }}
        POSTGRES_DB: tododb
      ports:
        - 5433:5432
      options: >-
        --health-cmd "pg_isready -U postgres -d tododb"
        --health-interval 10s
        --health-timeout 5s
        --health-retries 5

  steps:
    - name: Initialize Database
      env:
        PGPASSWORD: ${ secrets.TEST_DB_PASSWORD }}
      run: |
        until pg_isready -h 127.0.0.1 -p 5433 -U postgres; do
          sleep 2
        done
        psql -h 127.0.0.1 -p 5433 -U postgres -d tododb -f
        ./database/database.sql

    - name: Run Tests
      env:
        DB_USER: postgres
        DB_PASSWORD: ${ secrets.TEST_DB_PASSWORD }}
```

```
DB_HOST: localhost
DB_PORT: 5433
DB_NAME: tododb
run: cd backend && npm test
```

- Cho phần Deploy

```
# -----
# JOB 3: DEPLOY TO VPS
# -----
deploy:
  name: Deploy to Production
  needs: build-and-push # Ensures we only deploy IF the build was
successful
  runs-on: ubuntu-latest

  # Only deploy when pushing to the main branch
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'

  steps:
    - name: Checkout Code
      uses: actions/checkout@v4

    # 2. SSH into the VPS and run Docker commands
    - name: Execute Deployment Commands via SSH
      uses: appleboy/ssh-action@master
      with:
        host: ${ secrets.HOST }
        username: ${ secrets.USERNAME }
        key: ${ secrets.SSH_PRIVATE_KEY }
        script: |
          cd /var/wwwrootdev1/devops-project-hcm-cos1203

          sudo git pull origin main

          sudo docker-compose pull

          sudo docker-compose up -d
```

Sử dụng Kubernetes (K8s)

etcd: Bổ sung 1 slide giới thiệu về database trong Kubernetes, cấu trúc dữ liệu và đặc thù về chức năng của chúng

Ví dụ kiểu dữ liệu Pairs of Key and Value (JSON, YAML, Text) về etcd

- Record 1: Địa chỉ máy chủ database
 - Key: /config/db/host
 - Value: "127.0.0.1"
- Record 2: Cổng kết nối PostgreSQL
 - Key: /config/db/port
 - Value: 5432
- Record 3: Token xác thực
 - Key: /services/auth/token
 - Value: "eyJhbGciOiJIUzI1NiIs..."
- Record 4: Trạng thái node trong cluster
 - Key: /cluster/nodes/node1
 - Value: {"ip":"10.0.0.1","status":"healthy"}
- Record 5: Bật/tắt tính năng UI mới
 - Key: /feature/flag/newUI
 - Value: true

Đặc điểm quan trọng khi triển khai thực tế

- Immutable updates: mỗi lần ghi dữ liệu sẽ tạo bản mới, không ghi đè trực tiếp.
- Multi-version concurrency control (MVCC): đảm bảo tính nhất quán trong môi trường phân tán.
- Watch API: cho phép ứng dụng đăng ký theo dõi thay đổi trên một key hoặc prefix.

SSH: Hướng dẫn các lệnh sử dụng ssh cơ bản với Terminal hoặc PowerShell (lưu ý nếu đã đăng nhập rồi thì không cần bắt đầu câu lệnh bởi ssh)

- Đăng nhập: `ssh <username>@<Server IP> -p <Server Port>`: Ví dụ:
`ssh root@103.1.236.26 -p 22`
- Password: Nhập mật khẩu với màn hình đen bất động
- Tạo account chuyên cho deployment.
- Liệt kê thư mục từ xa: `ssh user@remote_host "ls -la /var/log"`
- Sao chép file: `scp file.txt user@remote_host:/home/user/`
- Copy key: `ssh-copy-id user@remote_host`
- Chuyển port tunnel: `ssh -L 8080:localhost:80 user@remote_host`
- Cho phép truy cập ngược từ remote về local: `ssh -R 8080:localhost:22 user@remote_host`
- Mount thư mục từ xa bằng SSHFS: `sshfs user@remote_host:/data /mnt/remote_data`
- Tạo Key pair (public and private): `ssh-keygen -t rsa -b 4096 "your-email@gmail.com"`
- Thêm SSH Key vào server đã tồn tại
 - Mở file `~/.ssh/authorized_keys` trên server bằng lệnh `vim ~/.ssh/authorized_keys`
 - Thêm nội dung của SSH public key vào file này và lưu lại.
Điều này sẽ cho phép bạn sử dụng private key để kết nối vào server mà không cần mật khẩu
- Kết nối SSH bằng SSH Key từ Ubuntu lên server

- Chuyển file private key (id_rsa) từ server về máy của bạn bằng lệnh scp
- `scp root@host:/root/.ssh/id_rsa /root/ssh.pem`
- Note: Thay root@host bằng thông tin của server của bạn và /root/ssh.pem bằng nơi bạn muốn lưu file private key trên máy của bạn
- Sửa file cấu hình SSH (/user/.ssh/config). Nếu file này chưa tồn tại, bạn có thể tạo mới và cấu hình như sau:
 - Host your_server_IP
 - User root
 - Port 22
 - PreferredAuthentications publickey
 - IdentityFile "/root/ssh.pem"
- Mở Port 22 để triển khai CI/CD với GitHub Actions nhưng bị báo lỗi Timeout dial tcp ***:22: i/o timeout. 1 số lệnh thao tác của ufw:
 - `sudo systemctl status ufw` => kiểm tra trạng thái của ufw đã được active hay chưa
 - `sudo ufw enable` => chạy lệnh active nếu trạng thái ufw là inactive
 - `sudo ufw allow 22/tcp` => mở port 22
 - `sudo ufw reload` => apply rule cho ufw
 - **Note:** hướng dẫn sử dụng UFW thông qua chatgpt, các công cụ tìm kiếm hoặc <https://kb.pavietnam.vn/cach-thiet-lap-tuong-lua-ufw-tren-ubuntu-20-04.html>

Quyền trong hệ điều hành Linux

- Cấu trúc bộ 3 quyền rwx với r là quyền đọc Read, w là quyền ghi Write, và x là quyền thực thi Execute. Nếu 1 file hay thư mục không có quyền nào đó thì vị trí của chúng được gán dấu gạch ngang (-). Ví dụ r-x ám chỉ tệp chỉ có quyền đọc và thực thi, không được phép ghi
- Ngoài ra có thể sử dụng số hệ octal để đại diện cho các nhóm quyền này

- 7 = rwx (đọc, ghi, thực thi)
- 5 = r-x (đọc, thực thi)
- 0 = --- (không quyền)
- Quy tắc phân quyền – là gỡ phân quyền trước đó, + là thêm quyền mới; nhóm người dùng gồm u là user hiện tại hoặc user sở hữu tệp, o là others, g là nhóm có liên quan đến user hiện tại.
 - `chmod u+x script.sh` # thêm quyền thực thi cho owner
 - `chmod o-r file.txt` # bỏ quyền đọc của others
 - `chmod g-w file.txt` # bỏ quyền ghi của group
- Change Owner:
 - Cấp quyền chỉnh sửa cho nhóm devteam chỉnh sửa thư mục /shared:
 - `sudo chown :devteam /shared`
 - `sudo chmod 770 /shared`
 - Chỉ cho admin đọc file config
 - `sudo chown admin:admin config.cfg`
 - `sudo chmod 400 config.cfg`
 - `sudo chown newuser:newgroup file.txt`

Deploy dự án lên ubuntu

- Cài pm2

5. Các lệnh phổ biến với kubectl

Lệnh	Ý nghĩa	Ví dụ
<code>kubectl get</code>	Liệt kê tài nguyên pod, node trong cluster	<code>kubectl get pods</code> (xem danh sách Pod)
<code>kubectl describe</code>	Hiển thị chi tiết tài nguyên	<code>kubectl describe pod my-pod</code>

kubectl logs	Xem log của Pod/Container	kubectl logs my-pod -c my-container
kubectl exec	Thực thi lệnh trong container	kubectl exec -it my-pod -- /bin/bash
kubectl apply -f	Triển khai cấu hình từ file YAML	kubectl apply -f deployment.yaml
kubectl delete	Xóa tài nguyên	kubectl delete pod my-pod
kubectl create	Tạo tài nguyên mới từ file hoặc trực tiếp	kubectl create deployment nginx -- image=nginx
kubectl scale	Thay đổi số lượng replica	kubectl scale deployment nginx -- replicas=3
kubectl rollout status	Kiểm tra trạng thái rollout của Deployment	kubectl rollout status deployment/nginx
kubectl config use-context	Chuyển đổi giữa các cluster/kontekst	kubectl config use-context my-cluster

Khởi tạo Master Node cho phép nhân bản từ 2 đến 10 Replica trong điều kiện CPU đã hoạt động 80% công suất như mô tả của slide: kubectl: kubectl autoscale deployment myapp --min=2 --max=10 --cpu-percent=80

6. Các lệnh phổ biến với **minikube** để test trên môi trường máy cục bộ

Lệnh	Ý nghĩa	Ví dụ
minikube start	Khởi động một cluster Kubernetes cục bộ	minikube start -- driver=docker
minikube stop	Dừng cluster đang chạy	minikube stop

minikube delete	Xoá cluster	minikube delete
minikube status	Kiểm tra trạng thái cluster	minikube status
minikube dashboard	Mở giao diện web Kubernetes Dashboard	minikube dashboard
minikube ip	Lấy địa chỉ IP của node Minikube	minikube ip
minikube logs	Xem log để debug cluster	minikube logs

7.